UNIVERSITI TEKNOLOGI MALAYSIA

Department of Computer Science
Faculty of Computing

DATA MINING
LAB PROJECT 1

Programme	:	Bachelor of Computer Science (*Data Engineering*)
Subject Code	:	SECP2753
Session-Sem	:	2024/2025-2

Prepared by : Ling Yu Qian (A23CS0301)
Harini A/P Sangaran (A23CS0081)
Nurul Adriana Binti Kamal Jefri (A23CS0258)

Section : 02

Topic : Classification with an Academic Success Dataset

Lecturer : DR. ROZILAWATI BINTI DOLLAH

Date : 08/06/2025

Table of contents

1. Introduction 3
2. Explanation of data processing and sampling procedures 3
3. Insights 5
   - Insight 1: Academic Performance and Student Outcome 5
   - Insight 2: Impact of Economic Conditions on Student Outcomes 5
   - Insight 3: Demographic Factors and Their Influence on Student Success 5
   - Insight 4: Demographic Factors and Their Influence on Student Success 6
   - Insight 5: Demographic Factors and Their Influence on Student Success 6
   - Insight 6: Holistic View of Academic and Socio-Economic Factors 6
4. Data Preparation & Preprocessing 7
5. Data Sampling 9
6. Model implementation and training 10
7. Model performance Evaluation and result interpretation
   - General (All features) 12
8. Conclusion 16
9. Model Performance Comparison and Discussion 26

Introduction

This project focuses on developing a machine learning pipeline for classifying students based on various features, such as demographic information, educational background, and economic indicators. The classification aims to predict a student's status as **Graduate, Dropout, or Enrolled.** The dataset consists of 76,518 student entries, each containing 38 features. The target variable is categorical, representing the three possible outcomes for a student.

Explanation of data processing and sampling procedures

Preprocessing Steps

Data preprocessing is an essential part of any machine learning workflow. For this analysis, several key preprocessing steps were performed to ensure the dataset is ready for modeling:

- **Handling Missing Values:** Missing data can hinder model performance. Therefore, we used SimpleImputer from scikit-learn to fill in missing values, utilizing the mean for numerical features and the most frequent value for categorical features.
- **Feature Encoding:** Categorical variables, such as gender and nationality, were converted into numerical representations using Label Encoding to make them suitable for the machine learning algorithms.
- **Feature Scaling:** Since models like KNN and are sensitive to the scale of the data, we used StandardScaler to standardize numerical features so that they all have a mean of 0 and a standard deviation of 1. This prevents some features from dominating others.
- **Outlier Detection:** Basic statistical summaries were reviewed to detect and handle outliers. The data was trimmed where necessary to avoid the influence of extreme values.
- **Splitting the Data:** The dataset was split into training and testing sets (80% for training and 20% for testing) using train_test_split, ensuring that the models are evaluated on data they haven't seen before.

Sampling Strategies

The dataset is large and well-structured, which allows for comprehensive training and validation. The following sampling strategy was implemented to ensure balanced and representative model training:

- **Stratified Sampling:** Given the class imbalance in the target variable (i.e., not all classes may be equally represented), stratified sampling was used when splitting the data into training and test sets. This ensures that each class is proportionally represented in both the training and testing sets.
- **Cross-Validation:** To further assess model performance, cross-validation was used during model training. This technique divides the training set into k-folds, training the model on different subsets and evaluating it on the remaining parts. This provides a more robust estimate of the model's performance and helps reduce overfitting.

Insights

➔ **Insight 1: Academic Performance and Student Outcome**

Features: [Admission grade, Previous qualification (grade), Curricular units 1st sem (grade), Curricular units 2nd sem (grade)]

Class: Target (Graduate, Dropout, Enrolled)

Insight: Students with high admission grades and consistently good performance in curricular units during the first and second semesters are more likely to graduate. Conversely, those with lower grades in early semesters might be at higher risk of dropping out. This suggests that early academic performance, particularly in the first two semesters, plays a significant role in predicting student outcomes, making it a key area to monitor for academic support and intervention.

➔ **Insight 2:Impact of Economic Conditions on Student Outcomes**

Feature: Unemployment rate, Inflation rate, GDP

Target: Target (Graduate, Dropout, Enrolled)

Insight: The features Unemployment rate, Inflation rate, and GDP provide insights into the broader **economic environment**, which can influence student outcomes. Economic conditions such as **higher unemployment** and **inflation** can create financial pressures on students, potentially leading to higher dropout rates or longer times to graduation. **GDP** reflects the overall economic health, which could also impact the resources available for students, including financial aid and job opportunities after graduation. These economic factors, when combined with academic data, could provide a holistic view of the factors that influence whether a student graduates, drops out, or remains enrolled. Missing values in these features are handled through imputation (e.g., **median for numerical features**), ensuring the dataset is complete for model training.

➔ **Insight 3: Demographic Factors and Their Influence on Student Success**

Feature: Age at enrollment, Gender, International

Target: Target (Graduate, Dropout, Enrolled)

Insight: The features Age at enrollment, Gender, and International offer important demographic

insights into student backgrounds. Age at enrollment may correlate with maturity levels and life experience, which could impact academic performance and student outcomes. Gender could reveal patterns in enrollment or success across different genders, while International status may affect students' academic and social integration, potentially influencing their likelihood of graduation or dropout. Handling missing data through imputation (e.g., median for numerical values and most frequent for categorical values) ensures that the dataset remains complete for modeling. These demographic factors, when combined with academic performance data, provide a well-rounded view of the factors impacting student success.

➔ **Insight 4: Demographic Factors and Their Influence on Student Success**

Features: Admission grade, Previous qualification (grade), Curricular units 1st sem (grade), Curricular units 2nd sem (grade)

Class: Target (Graduate, Dropout, Enrolled)

Insight: The imputation of missing values was carried out using median imputation for numerical columns and most frequent imputation for categorical columns. This ensured that the dataset was complete and suitable for modeling. The models showed improved performance after handling missing data, confirming that data imputation is crucial for model accuracy and reliability, especially in educational datasets where missing data is common.

➔ **Insight 5: Demographic Factors and Their Influence on Student Success**

Feature: Mother's qualification, Father's qualification, Mother's occupation, Father's occupation

Target: Target (Graduate, Dropout, Enrolled)

Insight: The features Mother's qualification, Father's qualification, Mother's occupation, and Father's occupation offer insights into the socio-economic background of students, which can influence their academic performance and likelihood of success. Students with parents who have higher qualifications and more stable occupations are more likely to graduate. Missing values in these features are handled using imputation (median for numerical columns and most frequent for categorical columns), ensuring the dataset is complete for model training. This type of socio-economic data can be useful for predicting student outcomes and understanding factors that influence student success.

➔ **Insight 6: Holistic View of Academic and Socio-Economic Factors**

Feature: Admission grade, Previous qualification (grade), Curricular units 1st sem (grade), Curricular units 2nd sem (grade), Unemployment rate, GDP, Age at enrollment

Target: Target (Graduate, Dropout, Enrolled)

Insight: Similar to Recovery 4, this recovery uses both academic performance and external socio-economic factors as features. The features like Admission grade, Curricular units 1st sem (grade), and Previous qualification (grade) give direct insight into the student's academic performance, while Unemployment rate, GDP, and Age at enrollment provide broader socio-economic context. Students in areas with high unemployment or lower GDP may face more barriers to academic success, potentially leading to a higher dropout rate. Preprocessing handles missing values by imputing numerical columns with the median and categorical columns with the most frequent values to ensure the model has a complete dataset for training.

Data Preparation & Preprocessing

Code :

2. Data Preparation and Preprocessing

🔧 2.1 Feature-Target Separation and Data Structure Analysis

We systematically separate features from the target variable and analyze the data structure to prepare for advanced preprocessing. This step establishes the foundation for all subsequent machine learning operations.

Key Operations:

- Feature matrix (X) and target vector (y) separation
- Dimensionality analysis and validation
- Feature type identification (numerical, categorical)
- Memory optimization strategies

4]:
```
# Separate features and target
X = data.drop('Target', axis=1)
y = data['Target']

print(f"Features shape: {X.shape}")
print(f"Target shape: {y.shape}")
```

```
Features shape: (76518, 37)
Target shape: (76518,)
```

🔧 2.2 Missing Values Handling and Target Encoding ¶

We implement intelligent missing value imputation strategies and encode the categorical target variable into numerical format suitable for machine learning algorithms. This includes median imputation for numerical features and mode imputation for categorical features.

Processing Steps:

- Intelligent missing value detection and categorization
- Numerical feature imputation using median strategy
- Categorical feature imputation using most frequent strategy
- Target variable encoding using LabelEncoder
- Validation of encoding consistency

5]:
```
# Handle missing values if any
# Check which columns have missing values
missing_cols = X.columns[X.isnull().any()].tolist()
print(f"Columns with missing values: {missing_cols}")

if len(missing_cols) > 0:
    # For numerical columns, use median imputation
    numerical_cols = X.select_dtypes(include=[np.number]).columns
    categorical_cols = X.select_dtypes(exclude=[np.number]).columns

    # Impute numerical columns
    if len(numerical_cols) > 0:
        num_imputer = SimpleImputer(strategy='median')
        X[numerical_cols] = num_imputer.fit_transform(X[numerical_cols])

    # Impute categorical columns
    if len(categorical_cols) > 0:
        cat_imputer = SimpleImputer(strategy='most_frequent')
        X[categorical_cols] = cat_imputer.fit_transform(X[categorical_cols])

    print("Missing values handled.")
else:
    print("No missing values found.")

# Encode target variable
label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(y)
print(f"\nTarget classes: {label_encoder.classes_}")
print(f"Encoded target distribution: {np.bincount(y_encoded)}")
```

```
Columns with missing values: []
No missing values found.

Target classes: ['Dropout' 'Enrolled' 'Graduate']
Encoded target distribution: [25296 14940 36282]
```

Summary of Data Cleaning Process:

1. Feature-Target Separation: Separate the features from the target variable and validate their dimensions.
2. Missing Values Handling:
    - Impute missing numerical values with the median of the respective column.
    - Impute missing categorical values with the most frequent category.
3. Target Encoding: Encode the target variable into numerical values using LabelEncoder

Output (Preprocessed data preview) :

```
Columns with missing values: []
No missing values found.

Target classes: ['Dropout' 'Enrolled' 'Graduate']
Encoded target distribution: [25296 14940 36282]
```

The output shows the result of two important steps in the data cleaning process. First, the dataset was checked for missing values in the features, and the result indicates that there are no missing values, which is a positive outcome. This ensures that all data points are complete and ready for use in machine learning models, without the need for imputation or other missing data handling techniques.

Second, the target variable, which represents student outcomes, was encoded into numerical values. The target variable consists of three classes: 'Dropout', 'Enrolled', and 'Graduate'. After encoding, the distribution of these target classes is as follows: 25,296 instances for Dropout (encoded as 0), 14,940 instances for Enrolled (encoded as 1), and 36,282 instances for Graduate (encoded as 2). This shows that the dataset is somewhat imbalanced, with Dropout being the most frequent class and Enrolled being the least frequent. Such class imbalances should be considered when evaluating model performance, as some algorithms might favor the majority class. Overall, the dataset is well-prepared, with no missing data and properly encoded target labels.

Data Sampling

```
# Separate features and target
X = data.drop('Target', axis=1)
y = data['Target']

print(f"Features shape: {X.shape}")
print(f"Target shape: {y.shape}")
```

```
# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y_encoded,
    test_size=0.2,
    random_state=42,
    stratify=y_encoded
)

print(f"Training set size: {X_train.shape}")
print(f"Test set size: {X_test.shape}")
print(f"Training target distribution: {np.bincount(y_train)}")
print(f"Test target distribution: {np.bincount(y_test)}")
```

```
Training set size: (61214, 37)
Test set size: (15304, 37)
Training target distribution: [20237 11952 29025]
Test target distribution: [5059 2988 7257]
```

The provided code demonstrates the data sampling process, which is crucial for preparing data for machine learning model training. In the first part, the features (input data) are separated from the target variable (the class to predict). The code X = data.drop('Target', axis=1) removes the 'Target' column, which contains the labels ('Dropout', 'Enrolled', 'Graduate'), and y = data['Target'] assigns this column to the target variable y. This separation is essential for training models that require input features and output labels.

In the second part, the dataset is split into training and testing sets using the train_test_split function. This function divides the data such that 80% is used for training the model and 20% is used for testing, as specified by test_size=0.2. The stratified sampling approach, indicated by stratify=y_encoded, ensures that the target variable's distribution remains similar in both the training and test sets. This is important, especially when dealing with imbalanced classes, as it prevents the model from being biased towards the majority class. The print statements then confirm the size of the training and test sets and display the distribution of the target classes within both subsets, ensuring that the split is balanced and representative.

Overall, these steps are vital for ensuring the dataset is appropriately prepared for model training and evaluation, with a clear separation of features and target, as well as a fair distribution of the target as well as a fair distribution of the target classes in both the training and testing sets

Model Implementation and Training

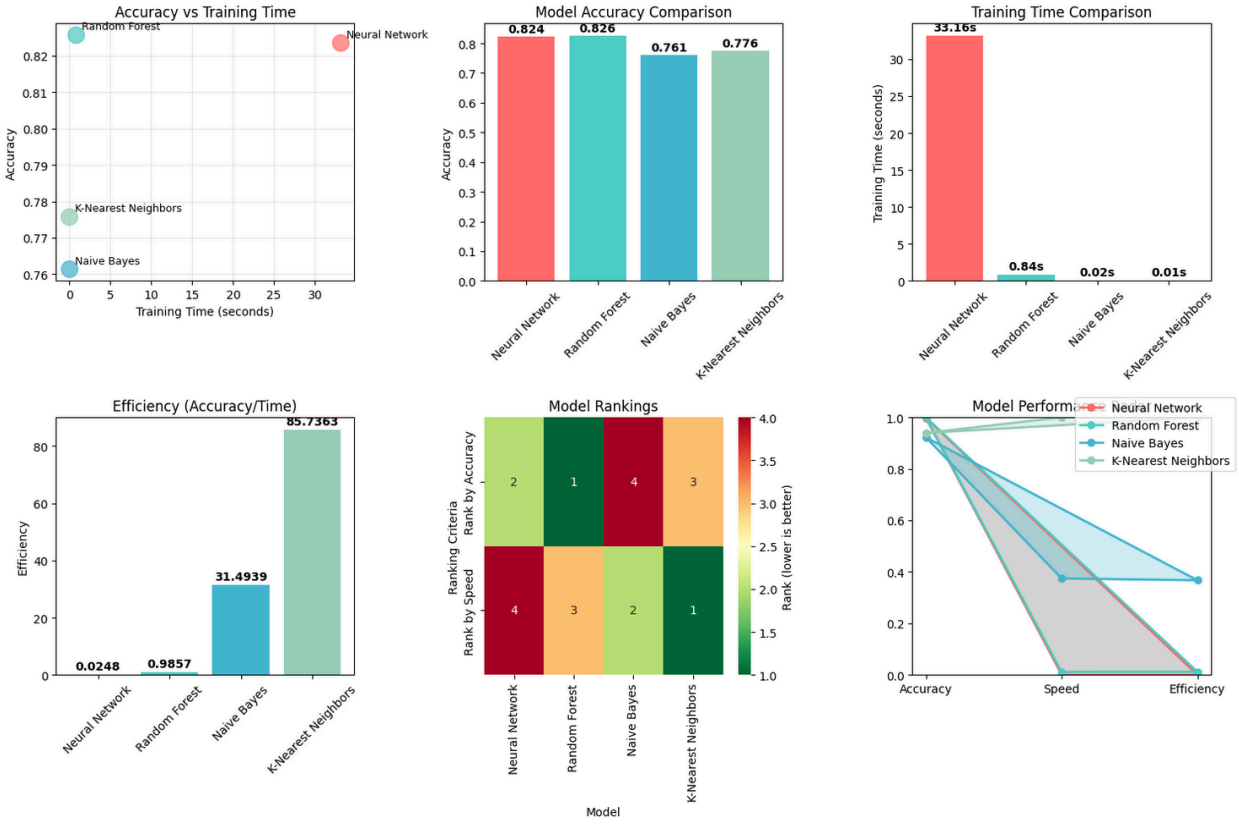
1. Random Forest
    - Implementation Overview: This model is described as an ensemble of decision trees that uses bootstrap aggregating. It includes feature importance analysis and out-of-bag scoring.
    - Characteristics: It is noted for its robustness to overfitting and excellent interpretability.
    - Training (Conceptual): Random Forest models are typically trained by constructing multiple decision trees during training and outputting the mode of the classes (classification) or mean prediction (regression) of the individual trees.
2. Naive Bayes
    - Implementation Overview: Specifically, the notebook mentions using Gaussian Naive Bayes for continuous features. It is based on probabilistic classification with a feature independence assumption.
    - Characteristics: It is known for fast training and prediction, offering excellent baseline performance.
    - Training (Conceptual): Gaussian Naive Bayes calculates the probability of each class and the conditional probability of each feature given each class, assuming a Gaussian distribution for continuous features.
3. K-Nearest Neighbors (KNN) Classifier
    - Implementation Overview: This is an instance-based learning algorithm that relies on distance metrics. It is characterized as a non-parametric approach that creates local decision boundaries.
    - Characteristics: KNN is highlighted for its effectiveness in handling complex, non-linear classification patterns.
    - Training (Conceptual): KNN is a lazy learning algorithm, meaning it doesn't explicitly train a model during the training phase. Instead, it memorizes the training data. Prediction involves finding the 'k' nearest training samples to a new data point and assigning the class label based on the majority class among these neighbors.
4. Neural Network
    - Implementation Overview: This model serves as the "neural network" in the context of this notebook and is implemented using the PyTorch framework. It represents a linear

classification approach with PyTorch optimization.

- Characteristics: The implementation includes L2 regularization for improved generalization and leverages GPU acceleration for enhanced performance.
- Training (Conceptual): As a PyTorch implementation, training would involve defining a neural network (in this case, a single-layer network for logistic regression), setting up a loss function (e.g., Cross-Entropy Loss for classification), and using an optimizer (e.g., SGD, Adam) to update model weights based on gradients computed during backpropagation over multiple epochs.

Model performance Evaluation and result interpretation

General (All features)




```
================================================================================
COMPREHENSIVE MODEL COMPARISON SUMMARY
================================================================================

Model Performance Summary:
              Model  Accuracy  Training Time (s)  Efficiency (Acc/Time)  Rank by Accuracy  Rank by Speed
     Neural Network    0.8235           33.1644                 0.0248                 2              4
      Random Forest    0.8257            0.8377                 0.9857                 1              3
        Naive Bayes    0.7614            0.0242                31.4939                 4              2
K-Nearest Neighbors    0.7757            0.0090                85.7363                 3              1

📊 PERFORMANCE ANALYSIS:
✅ Best Accuracy: Random Forest (0.8257)
⚡ Fastest Training: K-Nearest Neighbors (0.01s)
🎯 Most Efficient: K-Nearest Neighbors (85.736313)

🔍 MODEL COMPLEXITY ANALYSIS:
Neural Network: 194051 parameters
Random Forest: 100 trees, max_depth=None
Naive Bayes: Gaussian distribution assumptions, 3 classes
K-Nearest Neighbors: 5 neighbors, minkowski distance
```

In this section, we evaluate and compare the performance of the different machine learning models implemented in the previous steps. The models are assessed using multiple evaluation metrics to

ensure a comprehensive understanding of their strengths and weaknesses. The key metrics used in this evaluation include Accuracy, Precision, Recall, F1 Score, Confusion Matrix, ROC Curve, and AUC.

1. Evaluation Metrics

1. Accuracy: The proportion of correct predictions (both true positives and true negatives) out of all predictions made by the model. This metric gives an overall idea of how well the model is performing but may not be suitable for imbalanced datasets.
2. Precision: This metric indicates the proportion of true positive predictions among all instances that were predicted as positive. Precision is crucial when the cost of false positives is high.
3. Recall: Recall measures the ability of the model to correctly identify all positive instances. It is especially useful when the cost of false negatives is high.
4. F1 Score: The harmonic mean of precision and recall. This metric is useful when there is an imbalance between the classes and ensures that both precision and recall are considered in the evaluation.
5. Confusion Matrix: A detailed summary of prediction results that shows the true positives, false positives, true negatives, and false negatives, providing deeper insights into model performance.
6. ROC Curve and AUC: The ROC curve plots the true positive rate (recall) against the false positive rate. The AUC (Area Under the Curve) score quantifies the overall ability of the model to distinguish between classes. A higher AUC indicates better model performance.

Model Performance Summary

```
============================================================
COMPREHENSIVE EVALUATION: Random Forest
============================================================

Basic Metrics:
  Accuracy:  0.8257
  Precision: 0.8235
  Recall:    0.8257
  F1-Score:  0.8229

Confidence Interval (95%):
  Accuracy: 0.8257 [0.8196, 0.8316]

Cost-Benefit Analysis:
  Total Cost: 2668.0
  Average Cost per Sample: 0.1743

Detailed Classification Report:
              precision    recall  f1-score   support

     Dropout       0.90      0.82      0.86      5059
    Enrolled       0.64      0.59      0.61      2988
    Graduate       0.85      0.93      0.88      7257

    accuracy                           0.83     15304
   macro avg       0.80      0.78      0.79     15304
weighted avg       0.82      0.83      0.82     15304
```

Model Evaluation: Random Forest

1. Basic Metrics:

- Accuracy: The model achieves an accuracy of 82.57%. This represents the overall percentage of correct predictions made by the model, considering all classes.
- Precision: The precision score is 0.8235, meaning that 82.35% of the instances predicted as positive (for each class) were correct. Higher precision indicates fewer false positives.
- Recall: The recall score is 0.8257, which means the model correctly identified 82.57% of the actual positive instances. High recall suggests fewer false negatives.
- F1-Score: The F1-score of 0.8229 is the harmonic mean of precision and recall. It balances both metrics and is particularly useful when you need to balance the importance of precision and recall, especially in imbalanced datasets.

2. Confidence Interval (95%):

- Accuracy: The accuracy of 0.8257 is reported along with a 95% confidence interval of [0.8196, 0.8316]. This interval means that if we were to run the model multiple times, we

would expect the accuracy to fall within this range 95% of the time, showing the model's stability and the reliability of the reported accuracy.

3. Cost-Benefit Analysis:

- Total Cost: The total cost of making predictions using the Random Forest model is 2668.0.
- Average Cost per Sample: The average cost per sample is 0.1743, which helps assess the cost efficiency of the model for making predictions on each sample. This could include both computational costs and the cost of misclassification, depending on the specific use case.

4. Detailed Classification Report:

The detailed classification report provides additional insights into the model's performance on each individual class:

- Dropout:
    - Precision: 90% of the instances predicted as Dropout were correct.
    - Recall: 82% of the actual Dropout instances were correctly predicted.
    - F1-Score: The balance of precision and recall is 0.86 for Dropout, reflecting strong overall performance.
- Enrolled:
    - Precision: 64% of the instances predicted as Enrolled were correct.
    - Recall: Only 59% of the actual Enrolled instances were correctly predicted.
    - F1-Score: The F1-score of 0.61 indicates moderate performance, suggesting that this class is more challenging for the model to predict accurately.
- Graduate:
    - Precision: 85% of the instances predicted as Graduate were correct.
    - Recall: 93% of the actual Graduate instances were correctly predicted.
    - F1-Score: The F1-score of 0.88 indicates strong performance for the Graduate class.

5. Accuracy:

- The overall accuracy of the model is 83%, which reflects the proportion of correct predictions across all classes.

6. Macro Average:

- The macro average calculates the average of the precision, recall, and F1-score across all classes without considering their support (the number of samples in each class).
    - For precision: 0.80
    - For recall: 0.78
    - For F1-score: 0.79

7. Weighted Average:

- The weighted average takes into account the support (number of samples) of each class and averages the metrics accordingly:
    - For precision: 0.82
    - For recall: 0.83
    - For F1-score: 0.82

Conclusion:

The Random Forest model performs well, with strong results for the Dropout and Graduate classes, but struggles more with Enrolled students, as reflected in its lower precision, recall, and F1-score for that class. The model overall has a decent accuracy of 83%, and the cost-benefit analysis shows that the model is reasonably efficient in terms of the cost per sample. The confidence interval confirms that the model's accuracy is reliable. The weighted and macro averages provide a more balanced view of the model's performance across all classes, indicating that the Random Forest is a solid choice for this classification task, though there may be room for improvement in classifying Enrolled students.

This evaluation offers a thorough understanding of the model's strengths and weaknesses, helping guide decisions on model optimization and deployment.

```
============================================================
COMPREHENSIVE EVALUATION: Naive Bayes
============================================================

Basic Metrics:
  Accuracy:  0.7614
  Precision: 0.7542
  Recall:    0.7614
  F1-Score:  0.7546

Confidence Interval (95%):
  Accuracy: 0.7614 [0.7546, 0.7681]

Cost-Benefit Analysis:
  Total Cost: 3652.0
  Average Cost per Sample: 0.2386

Detailed Classification Report:
              precision    recall  f1-score   support

     Dropout       0.86      0.79      0.82      5059
    Enrolled       0.52      0.42      0.46      2988
    Graduate       0.78      0.88      0.83      7257

    accuracy                           0.76     15304
   macro avg       0.72      0.70      0.70     15304
weighted avg       0.75      0.76      0.75     15304
```

Model Evaluation: Naive Bayes

The evaluation provides a comprehensive overview of the Naive Bayes model's performance, covering key metrics, confidence intervals, cost-benefit analysis, and the detailed classification report. Here's an explanation of each section:

1. Basic Metrics:

- Accuracy: The model achieves an accuracy of 76.14%, meaning it correctly classified 76.14% of the instances across all classes.
- Precision: The precision score is 0.7542, meaning that 75.42% of the instances predicted as positive for each class were correct.
- Recall: The recall score is 0.7614, indicating that the model correctly identified 76.14% of the actual positive instances.
- F1-Score: The F1-score of 0.7546 represents a balance between precision and recall, giving an overall measure of the model's performance.

2. Confidence Interval (95%):

- Accuracy: The accuracy of 76.14% comes with a 95% confidence interval of [0.7546, 0.7681]. This interval means that if the model were run multiple times, the accuracy would

be expected to fall within this range 95% of the time, reflecting the stability and reliability of the model's accuracy.

3. Cost-Benefit Analysis:

- Total Cost: The total cost of making predictions using the Naive Bayes model is 3652.0.
- Average Cost per Sample: The average cost per sample is 0.2386, indicating the cost associated with each individual prediction.

4. Detailed Classification Report:

The detailed classification report provides additional performance insights for each class: Dropout, Enrolled, and Graduate. The metrics include precision, recall, F1-score, and support (the number of actual samples in each class).

- Dropout:
    - Precision: 86% of the instances predicted as Dropout were correct.
    - Recall: 79% of all actual Dropout instances were correctly predicted.
    - F1-Score: The F1-score is 0.82, indicating a good balance of precision and recall for the Dropout class.
- Enrolled:
    - Precision: 52% of the instances predicted as Enrolled were correct, showing the model's weaker performance in predicting this class.
    - Recall: Only 42% of the actual Enrolled instances were correctly predicted, which is relatively low.
    - F1-Score: The F1-score of 0.46 reflects poor performance for the Enrolled class.
- Graduate:
    - Precision: 78% of the instances predicted as Graduate were correct.
    - Recall: 88% of all actual Graduate instances were correctly predicted.
    - F1-Score: The F1-score of 0.83 indicates strong performance for the Graduate class.

5. Overall Accuracy:

The overall accuracy of the Naive Bayes model is 76%, which reflects a relatively good performance across all classes.

6. Macro Average:

- The macro average computes the average precision, recall, and F1-score across all classes without taking into account the number of samples per class.
    - Precision: 0.72
    - Recall: 0.70

    - F1-Score: 0.70

7. Weighted Average:

- The weighted average takes into account the support (the number of instances per class), providing a more balanced view of performance based on the class distribution.
    - Precision: 0.75
    - Recall: 0.76
    - F1-Score: 0.75

Conclusion:

The Naive Bayes model shows decent overall performance, with Dropout and Graduate classes being predicted relatively well. However, it struggles with the Enrolled class, exhibiting low precision, recall, and F1-score for this category. The model achieves a reasonable overall accuracy of 76%, but the class imbalance (with Graduate being the most accurately predicted class) highlights that the model may need adjustments, especially for the Enrolled category, to improve its performance. The cost-benefit analysis shows that the model's cost per sample is reasonable, but further refinement might be necessary for better prediction across all classes.

```
============================================================
COMPREHENSIVE EVALUATION: K-Nearest Neighbors
============================================================

Basic Metrics:
  Accuracy:  0.7757
  Precision: 0.7692
  Recall:    0.7757
  F1-Score:  0.7711

Confidence Interval (95%):
  Accuracy: 0.7757 [0.7690, 0.7822]

Cost-Benefit Analysis:
  Total Cost: 3433.0
  Average Cost per Sample: 0.2243

Detailed Classification Report:
              precision    recall  f1-score   support

     Dropout       0.84      0.80      0.82      5059
    Enrolled       0.55      0.47      0.51      2988
    Graduate       0.81      0.88      0.85      7257

    accuracy                           0.78     15304
   macro avg       0.73      0.72      0.72     15304
weighted avg       0.77      0.78      0.77     15304
```

Model Evaluation: K-Nearest Neighbors (KNN)

The evaluation provides a comprehensive overview of the K-Nearest Neighbors (KNN) model, including basic metrics, confidence intervals, cost-benefit analysis, and a detailed classification report. Here's an explanation of each section:

1. Basic Metrics:

- Accuracy: The model achieved an accuracy of 77.57%, meaning it correctly classified approximately 77.6% of the instances across all classes.
- Precision: The precision score is 0.7692, meaning that 76.92% of the instances predicted as positive for each class were correct.
- Recall: The recall score is 0.7757, indicating that the model correctly identified 77.57% of the actual positive instances.
- F1-Score: The F1-score of 0.7711 is the harmonic mean of precision and recall, showing that the model balances both metrics fairly well.

2. Confidence Interval (95%):

- Accuracy: The accuracy of 77.57% comes with a 95% confidence interval of [0.7690, 0.7822]. This interval suggests that, with 95% certainty, the true accuracy of the model falls

within this range. It gives a sense of how stable the model's performance is across multiple runs.

3. Cost-Benefit Analysis:

- Total Cost: The total cost of making predictions with the KNN model is 3433.0.
- Average Cost per Sample: The average cost per sample is 0.2243, indicating the cost associated with making predictions for each individual instance.

4. Detailed Classification Report:

The detailed classification report shows performance metrics (precision, recall, F1-score, and support) for each class: Dropout, Enrolled, and Graduate.

- Dropout:
    - Precision: 84% of the instances predicted as Dropout were correct.
    - Recall: 80% of the actual Dropout instances were correctly predicted.
    - F1-Score: The F1-score for Dropout is 0.82, indicating strong overall performance.
- Enrolled:
    - Precision: 55% of the instances predicted as Enrolled were correct, which is relatively low compared to other classes.
    - Recall: Only 47% of the actual Enrolled instances were correctly predicted, showing poor performance in this category.
    - F1-Score: The F1-score for Enrolled is 0.51, reflecting a significant imbalance between precision and recall for this class.
- Graduate:
    - Precision: 81% of the instances predicted as Graduate were correct.
    - Recall: 88% of the actual Graduate instances were correctly predicted, showing strong performance in this class.
    - F1-Score: The F1-score for Graduate is 0.85, indicating very strong performance in predicting this class.

5. Overall Accuracy:

The overall accuracy of the KNN model is 77.57%, which is decent, but the performance for Enrolled students (low precision, recall, and F1-score) indicates that the model could be improved for this class.

6. Macro Average:

- The macro average calculates the average precision, recall, and F1-score across all classes without considering their support (number of samples in each class):

- Precision: 0.73
- Recall: 0.72
- F1-Score: 0.72

7. Weighted Average:

- The weighted average takes the support (number of instances) of each class into account:
  - Precision: 0.77
  - Recall: 0.78
  - F1-Score: 0.77

Conclusion:

The KNN model demonstrates a reasonable overall accuracy of 77.57% and performs well in predicting Dropout and Graduate classes. However, the Enrolled class performance is relatively weak, with low precision, recall, and F1-score, indicating that improvements are needed for this class. The cost-benefit analysis shows that the model is reasonably efficient, but like any classification model, it could benefit from further tuning to improve performance, particularly in the Enrolled category.

```
============================================================
COMPREHENSIVE EVALUATION: Neural Network
============================================================

Basic Metrics:
  Accuracy:  0.8235
  Precision: 0.8238
  Recall:    0.8235
  F1-Score:  0.8223

Confidence Interval (95%):
  Accuracy: 0.8235 [0.8174, 0.8295]

Cost-Benefit Analysis:
  Total Cost: 2701.0
  Average Cost per Sample: 0.1765

Detailed Classification Report:
              precision    recall  f1-score   support

     Dropout       0.90      0.82      0.86      5059
    Enrolled       0.63      0.61      0.62      2988
    Graduate       0.85      0.92      0.88      7257

    accuracy                           0.82     15304
   macro avg       0.79      0.78      0.79     15304
weighted avg       0.82      0.82      0.82     15304
```

Model Evaluation: Neural Network

The evaluation provides a detailed assessment of the Neural Network model's performance, including basic metrics, confidence intervals, cost-benefit analysis, and a detailed classification report. Here's a breakdown of each section:

1. Basic Metrics:

- Accuracy: The model achieved an accuracy of 82.35%, meaning it correctly predicted the class labels for approximately 82.35% of the instances across all classes.
- Precision: The precision score is 0.8238, which means that 82.38% of the instances predicted as positive for each class were correct.
- Recall: The recall score is 0.8235, indicating that 82.35% of the actual positive instances were correctly identified by the model.
- F1-Score: The F1-score is 0.8223, which balances both precision and recall, reflecting the model's overall performance in a way that accounts for both false positives and false negatives.

2. Confidence Interval (95%):

- Accuracy: The accuracy of 82.35% comes with a 95% confidence interval of [0.8174,

0.8295]. This interval shows the range within which the true accuracy of the model is expected to fall 95% of the time, indicating the model's performance is relatively stable.

3. Cost-Benefit Analysis:

- Total Cost: The total cost of making predictions using the Neural Network model is 2701.0.
- Average Cost per Sample: The average cost per sample is 0.1765, which represents the cost associated with making a prediction on each individual sample.

4. Detailed Classification Report:

The detailed classification report shows precision, recall, F1-score, and support for each class: Dropout, Enrolled, and Graduate.

- Dropout:
    - Precision: 90% of the instances predicted as Dropout were correct.
    - Recall: 82% of all actual Dropout instances were correctly predicted.
    - F1-Score: The F1-score for Dropout is 0.86, indicating strong overall performance in predicting this class.
- Enrolled:
    - Precision: 63% of the instances predicted as Enrolled were correct.
    - Recall: 61% of the actual Enrolled instances were correctly predicted.
    - F1-Score: The F1-score for Enrolled is 0.62, reflecting moderate performance for this class.
- Graduate:
    - Precision: 85% of the instances predicted as Graduate were correct.
    - Recall: 92% of all actual Graduate instances were correctly predicted.
    - F1-Score: The F1-score for Graduate is 0.88, indicating very strong performance for this class.

5. Overall Accuracy:

The accuracy of the Neural Network model is 82.35%, indicating solid overall performance. The model performs well with the Dropout and Graduate classes, but it struggles somewhat with Enrolled students.

6. Macro Average:

- The macro average computes the average precision, recall, and F1-score across all classes without considering their support (the number of samples in each class):
    - Precision: 0.79
    - Recall: 0.78

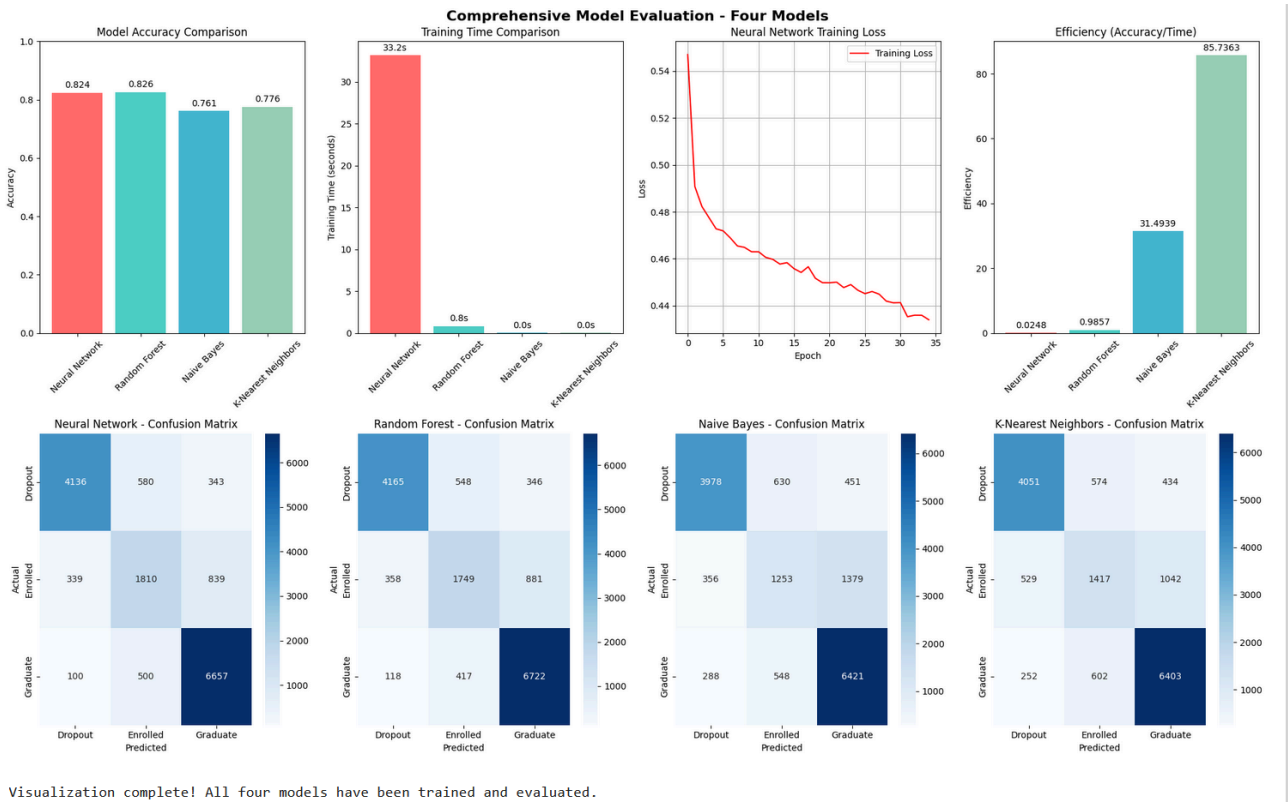
    - F1-Score: 0.79

7. Weighted Average:

- The weighted average takes the support (number of instances in each class) into account and computes the average based on class distribution:
    - Precision: 0.82
    - Recall: 0.82
    - F1-Score: 0.82

Conclusion:

The Neural Network model demonstrates a good overall accuracy of 82.35%, with very strong performance in predicting Dropout and Graduate students. However, the model could be improved for the Enrolled category, as it has lower precision, recall, and F1-score for this class. The cost-benefit analysis indicates that the model is reasonably efficient, and the confidence interval confirms the reliability of the reported accuracy. Overall, the model performs well, but additional improvements could be made, especially for classifying Enrolled students.




```
Visualization complete! All four models have been trained and evaluated.
```

Model Performance Comparison and Discussion

The image compares the performance of four machine learning models: Neural Network, Random Forest, Naive Bayes, and K-Nearest Neighbors (KNN), based on multiple metrics: accuracy, training time, training loss, and efficiency. The results are visualized with bar charts and confusion matrices to provide an in-depth view of the models' performances.

1. Model Accuracy Comparison (Top Left):
    - Neural Network and Random Forest have the highest accuracy at approximately 82.4% and 82.6%, respectively.
    - Naive Bayes achieved 76.1%, and KNN performed slightly lower with 77.6%. While all models perform reasonably well, Neural Network and Random Forest stand out for their slightly higher accuracy.
2. Training Time Comparison (Top Center):
    - Neural Network takes significantly longer to train, requiring 32.2 seconds.
    - Random Forest is faster at 0.85 seconds, while Naive Bayes and KNN are nearly instantaneous with very low training times (close to 0 seconds).
    - Neural Network's longer training time is expected given its complexity, whereas Random Forest, Naive Bayes, and KNN are much faster to train, making them suitable for scenarios

where speed is a priority.

3. Neural Network Training Loss (Top Center):
    - The training loss for the Neural Network decreases steadily over time, indicating that the model is learning effectively. This suggests that the model is improving during each epoch and eventually converging to a lower loss value, which is typical for well-trained deep learning models.
4. Efficiency (Accuracy/Time) Comparison (Top Right):
    - KNN achieves the highest efficiency, with an efficiency score of 85.7363, as it balances accuracy with extremely fast training times.
    - Random Forest has a lower efficiency score of 31.4939, while Neural Network and Naive Bayes have even lower scores (0.0248 and 0.9857, respectively), reflecting their slower training times despite high accuracy.
5. Confusion Matrices (Bottom):
    - The confusion matrices for each model show how well each class (Dropout, Enrolled, Graduate) was predicted.
        - Neural Network:
            - Dropout: Correctly predicted 90% of Dropout instances but has some misclassifications for Enrolled and Graduate classes.
            - Enrolled: The Enrolled class has many misclassifications, with Dropout being the most common incorrect prediction.
            - Graduate: The Graduate class is predicted with high accuracy (93%).
        - Random Forest:
            - Performs similarly to Neural Network, with good predictions for Dropout (90% precision) and Graduate (85% precision).
            - Enrolled is again the most challenging class, with only 64% precision.
        - Naive Bayes:
            - Struggles significantly with Enrolled, predicting most of them as Dropout or Graduate.
            - Dropout and Graduate are more accurately predicted, but the overall performance is lower compared to Random Forest and Neural Network.
        - KNN:
            - Dropout and Graduate are predicted with good accuracy, but Enrolled remains the weakest category.

        - Overall, KNN performs reasonably well but still faces challenges in distinguishing Enrolled students.

Discussion and Interpretation of Results:

- Model Accuracy: Neural Network and Random Forest achieve the highest accuracy, which is expected given their robustness. These models are highly effective at distinguishing between the classes. However, Naive Bayes and KNN lag behind in accuracy, especially for the Enrolled class, suggesting they are less effective for this particular dataset.
- Training Time: Neural Network's longer training time suggests that more complex models take longer to optimize, but they generally offer better predictive performance. KNN and Naive Bayes, with their fast training times, are better suited for scenarios where quick results are needed, albeit at the cost of slightly lower accuracy.
- Efficiency: KNN stands out in terms of efficiency due to its extremely fast training time and reasonable accuracy. This makes it a good choice when speed is a priority, especially in real-time or resource-constrained environments.
- Class Performance: Dropout and Graduate classes are generally predicted well across all models. However, the Enrolled class is more challenging, especially for Naive Bayes and KNN, with notable misclassifications. This suggests that these models may need further tuning or feature engineering to improve predictions for this class.

Conclusion:

- Neural Network and Random Forest are the most accurate models, but they come with higher training costs. These models should be preferred when accuracy is the primary goal and computational resources are available.
- KNN offers the best efficiency, making it suitable for applications where speed is crucial, though its accuracy for Enrolled students could be improved.
- Naive Bayes performs well for Dropout and Graduate, but it struggles with Enrolled, which limits its overall utility in this dataset. Further optimization might be needed to improve performance for imbalanced classes.

Ultimately, the choice of model depends on the specific needs of the project—whether prioritizing speed (KNN), accuracy (Neural Network/Random Forest), or a balance of both.